

# WALKTHROUGH DOCUMENT

## Multi-Role Hiring Automation System

*n8n (Docker) | Groq AI | Google Sheets | Gmail | Google Calendar*

|                 |                                                                                                         |
|-----------------|---------------------------------------------------------------------------------------------------------|
| Submitted By    | Zaki Haider                                                                                             |
| Email           | zakihaider7860@gmail.com                                                                                |
| Assignment      | n8n Automation Developer — Practical Assignment                                                         |
| Company         | Trilles AI                                                                                              |
| Date            | May 2026                                                                                                |
| Tools Used      | n8n (Self-Hosted Docker) + Groq AI (Llama 3.3) + Google Sheets + Gmail + Google Calendar + Google Drive |
| Roles Automated | Software Engineer (SWE) + Business Development Manager (BDM)                                            |

*This document explains the architecture, design decisions, assumptions, limitations, and future improvements of the Multi-Role Hiring Automation System built for the Trilles AI practical assignment.*

# 1. Executive Summary

The Multi-Role Hiring Automation System is a fully automated end-to-end recruitment pipeline built using n8n, designed to eliminate manual effort from the hiring process at a growing company. The company was actively hiring for two distinct roles — Software Engineer (SWE) and Business Development Manager (BDM) — but faced a critical bottleneck: every application was being reviewed manually, leading to hours of lost productivity, inconsistent evaluation standards, and candidates waiting days without any response.

This system solves that problem by automating the entire pipeline — from the moment an application arrives to the moment the candidate receives a response. Applications from both role-specific Google Sheets are ingested automatically, resumes are downloaded from Google Drive and processed by an AI model (Groq AI with Llama 3.3-70B), and each candidate is evaluated using role-specific criteria to produce a structured assessment including a score, classification (strong, average, or weak), summary, strengths, and concerns. Based on this AI assessment, the system automatically sends personalized interview invitations with Google Calendar events for strong candidates, notifies the hiring manager for average candidates requiring human judgment, and sends professional rejection emails to weak candidates — all without any human intervention.

The approach was to build two separate, independently scheduled workflows — one for processing and one for communication — ensuring clean separation of concerns, resilience, and exactly meeting the assignment's requirement that email sending runs on its own schedule independently of the main processing pipeline.

# 2. Architecture Overview

## Two-Workflow Design

The system is intentionally split into two completely separate n8n workflows, each running on its own independent schedule. This was a deliberate architectural decision, not an accident.

| Workflow | Name                      | Schedule         | Purpose                                |
|----------|---------------------------|------------------|----------------------------------------|
| 1        | Main Processing Pipeline  | Every 15 minutes | Ingest, screen, and store applications |
| 2        | Email & Calendar Pipeline | Every 30 minutes | Send emails and create calendar events |

## Workflow 1 — Main Processing Pipeline

The processing pipeline follows this exact sequence:

- Schedule Trigger (every 15 min) fires the workflow
- Read SWE Applications from Google Sheets — filtered to only unprocessed rows (Status = blank)

- Read BDM Applications from Google Sheets — same filter applied
- Tag each stream with role name (Software Engineer / Business Development Manager)
- Merge both streams into one unified flow
- Check if there are any applications — exit gracefully if none
- Download resume PDF from Google Drive using OAuth2
- Extract text from the PDF using Extract From File node
- Send resume text to Groq AI (Llama 3.3-70B) with role-specific evaluation prompt
- Parse the structured JSON response from AI
- Append results to Master Candidates Google Sheet
- Mark original application as "Processed" in SWE or BDM sheet (prevents reprocessing)

## Workflow 2 — Email & Calendar Pipeline

- Schedule Trigger (every 30 min) fires independently
- Read Master Candidates sheet — filtered to Email Status = "pending"
- Check if there are pending candidates — exit gracefully if none
- Switch node routes each candidate based on AI Classification
- Strong candidates: receive interview invitation email + Google Calendar event is created
- Average candidates: hiring manager receives notification email with full AI assessment
- Weak candidates: receive professional, encouraging rejection email
- Update Email Status to "sent" in Master sheet after each email

## Why Two Separate Workflows?

*The assignment specification explicitly states: "The email sending should not happen inline with the main processing workflow. It should run on its own schedule." This two-workflow design directly satisfies that requirement while also providing additional benefits: if email sending fails, it does not affect application processing; both workflows can be scaled independently; and the email pipeline can be manually triggered for urgent cases without re-running the entire processing pipeline.*

### 3. Key Design Decisions

#### Google Sheets as Central Database

Google Sheets was chosen as the central data store for three reasons: it is completely free with no setup overhead, it integrates natively with n8n through built-in nodes requiring no additional configuration, and it provides an instant visual interface for the hiring team to view candidate data without building a separate dashboard. The Master Candidates sheet acts as the single source of truth, storing every processed candidate with their full AI assessment.

#### Groq AI with Llama 3.3-70B

Groq was selected over other AI providers because it offers a completely free API tier with extremely fast inference speeds. The Llama 3.3-70B model provides high-quality structured analysis suitable for resume evaluation. Since Groq's API is compatible with the OpenAI API format, it was integrated using n8n's built-in OpenAI node with a custom Base URL (<https://api.groq.com/openai/v1>) — requiring zero additional setup or community nodes.

#### Role-Specific AI Prompts

A single generic AI prompt would produce unfair and inaccurate evaluations. The system uses a role-specific system prompt that instructs the AI to evaluate Software Engineers on technical criteria (programming languages, projects, GitHub activity, algorithms) and Business Development Managers on commercial criteria (sales targets, revenue generation, relationship building, industry knowledge). This ensures each candidate is judged fairly against the actual requirements of their applied role.

#### Status Column for Idempotency

Both the SWE and BDM application sheets include a Status column. When the workflow processes an application, it writes "Processed" to this column. The Google Sheets read nodes filter for rows where Status is blank — meaning the workflow will never process the same application twice, even if it runs hundreds of times. This makes the system completely safe to run continuously.

#### Google Drive Service Account for Resume Download

Resume files uploaded via Google Forms are stored in Google Drive. Accessing them required setting up a Google Cloud Service Account with Google Drive API access, sharing each uploaded file with the service account email, and using n8n's Google Drive node with the service account credential. This approach is more reliable than OAuth2 for automated file access as it does not require user interaction or token refresh management.

## Structured JSON from AI

The AI is explicitly instructed to return ONLY a valid JSON object with predefined fields: score, classification, summary, strengths, concerns, and interview\_recommended. A Code node then parses this JSON with error handling — if parsing fails for any reason, the candidate is classified as "average" and sent for manual review rather than crashing the workflow.

## 4. Assumptions Made

The following assumptions were made during the design and build of this system:

### **Resume File Format:**

Resumes uploaded via Google Form are stored as PDF or Word documents in Google Drive. The Google Drive node with Extract From File can handle both formats. Non-standard formats (e.g., scanned images without OCR) may result in empty text extraction.

### **One Sheet Per Role:**

Each role has exactly one Google Sheet receiving form responses. The system is designed for exactly two roles (SWE and BDM) with their respective sheets. Adding more roles would require extending the workflow with additional read nodes.

### **Google Form Column Names:**

The column names in both sheets follow a consistent format. The Tag nodes map raw column names to standardized field names (name, email, resumeUrl). Any change to Google Form questions would require updating the field mappings in the Tag nodes.

### **AI Response Format:**

The Groq AI model reliably returns valid JSON in the requested format. A fallback error handler is included for cases where parsing fails, but the system assumes the AI will generally follow the structured output instruction.

### **Email Deliverability:**

Gmail is used for sending emails. The system assumes the Gmail account used has sufficient daily sending quota (500 emails/day for free accounts) and that candidate email addresses provided in the application forms are valid and active.

### **Calendar Access:**

The Google Calendar connected to n8n belongs to the hiring team and has permission to invite external attendees. Interview slots are scheduled 3 days in advance at 10 AM by default — in production, this would be replaced with an actual availability checking system.

### **Single Run Environment:**

The system runs on a single n8n instance. In a production environment with high application volumes, the workflow would need to be distributed across multiple instances with proper queue management.

## 5. Known Limitations

### Google Drive File Access:

The current implementation requires each uploaded resume to be shared with the service account email. In a real Google Forms setup, files uploaded by applicants are stored in a specific Drive folder that needs to be shared with the service account once — making this a one-time setup rather than a per-file operation.

### Gmail Sending Limits:

Free Gmail accounts are limited to 500 emails per day. For high-volume hiring (hundreds of applications daily), this system would need to migrate to Google Workspace (2,000/day) or a dedicated email service like SendGrid or Mailgun.

### AI Bias and Consistency:

AI models can exhibit biases based on their training data. Resume screening by AI may inadvertently disadvantage candidates from certain backgrounds or with non-traditional career paths. The "average" classification routing to human review is intentionally designed as a safety net for borderline cases.

### No Real-Time Processing:

The system uses polling (checking every 15 minutes) rather than webhooks. This means there is up to a 15-minute delay between application submission and processing. For very time-sensitive hiring, a webhook-based trigger connected directly to Google Forms would be more appropriate.

### Interview Scheduling:

The current implementation creates a calendar event 3 days from processing time at 10 AM without checking actual availability. A production system would integrate with a scheduling tool like Calendly or check the hiring manager's actual calendar availability before proposing times.

### Single Language Support:

The AI prompt and email templates are in English only. International candidates submitting resumes in other languages may receive lower scores due to the AI's language processing limitations.

### No Candidate Portal:

Candidates have no way to check their application status. A more complete system would include a candidate-facing portal or automated status update emails at each stage of the process.

## 6. What I Would Improve With More Time

### 1. Proper Database (PostgreSQL or Airtable)

Replace Google Sheets as the central store with a proper relational database like PostgreSQL or Airtable. This would enable complex queries, better indexing, relationships between tables (candidates, applications, interviews), and much faster read/write performance at scale.

### 2. Webhook-Based Triggers

Replace the polling schedule trigger with a webhook that fires instantly when a new Google Form submission arrives. This would reduce processing delay from up to 15 minutes to under 30 seconds, dramatically improving candidate experience.

### 3. Intelligent Interview Scheduling

Integrate with Google Calendar's free/busy API or a scheduling platform like Calendly to automatically find available interview slots based on the hiring manager's actual calendar, then let the candidate self-select a time from available options.

### 4. Candidate Self-Service Portal

Build a simple web interface where candidates can track their application status in real time. This would reduce support emails and improve the employer brand by providing transparency throughout the process.

### 5. Multi-Stage Pipeline

Extend the system beyond initial screening to support multiple interview rounds, technical assessments, and offer management — creating a complete ATS (Applicant Tracking System) rather than just initial screening.

### 6. Analytics Dashboard

Build a Looker Studio or Metabase dashboard connected to the Master Candidates sheet showing metrics like applications per role, average AI scores, classification distribution, time-to-response, and conversion rates through the funnel.

### 7. Error Alerting

Add a dedicated error handling branch to every node that sends an immediate Slack or email notification to the system administrator if any part of the pipeline fails, with the full error details and the affected candidate's information.

### 8. Resume Parsing Improvements

Add support for multiple resume formats including LinkedIn PDF exports, Europass CVs, and image-based resumes (using OCR). Also implement a fallback that uses the AI to extract structured information even from poorly formatted resumes.

## 7. Error Handling & Edge Cases

How the System Handles Problems:

| Scenario                         | How It Is Handled                                                                                                                            |
|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| No new applications              | IF node checks item count. If zero, workflow exits gracefully with no errors and no unnecessary processing.                                  |
| AI returns malformed JSON        | Code node wraps JSON.parse in try/catch. On failure, candidate is classified as "average" and routed for manual human review.                |
| Same application processed twice | Status column filter in Google Sheets read node ensures only blank-status rows are read. Once marked "Processed", never reprocessed.         |
| Resume file inaccessible         | Google Drive node returns an error. The workflow stops for that item. The Status column remains blank so it will be retried on the next run. |
| Email sending fails              | Email Status remains "pending" in Master sheet. The email pipeline will attempt to send again on its next scheduled run (every 30 minutes).  |
| Multiple candidates in one run   | n8n processes all items in the batch automatically. Each item flows through the entire pipeline independently without affecting others.      |